Euskal Herriko Unibertsitatea

Software Ingeniaritza II

Diseinu patroiak

Simple Factory, Observer, Adapter eta Iterator

Taldekideak:

Izen-abizenak: Iago Vazquez

Izen-abizenak: Pablo Maretrínez

Entregatze-data: 6/11/2026

Gaien Aurkibidea

1 Simple Factory **2**
- 1.1 UML diseinua . . . 2
- 1.2 Diseinuan egindako aldaketak . . . 2
- 1.3 Inplementazioa: "mareos"sintoma berria gehitzea . . . 2
- 1.4 Symptom objektuen bakartasuna . . . 3

2 Observer patroia **3**
- 2.1 Lehen prototipoa . . . 3
  - 2.1.1 Observer patroiaren inplementazioa . . . 3
- 2.2 Bigarren prototipoa . . . 4
  - 2.2.1 PacientThermometerGUI klasearen inplementazioa . . . 4
  - 2.2.2 UML diagrama hedatua . . . 4

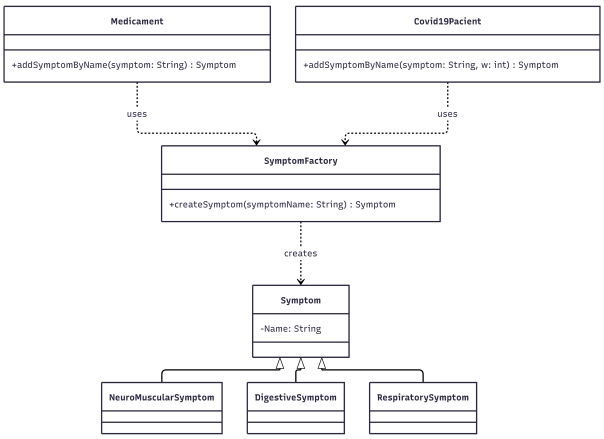
3 Adapter patroia **4**
- 3.1 Covid19PacientTableModelAdapter egokitzailea . . . 4
  - 3.1.1 Inplementazioa . . . 4
  - 3.1.2 Emaitzaren egiaztapena . . . 5
- 3.2 Hainbat pazienteren bistaratzea . . . 5
- 3.3 Taulen eguneratze automatikoa (hautazkoa) . . . 5

4 Adapter eta Iterator patroiak **6**
- 4.1 UML diseinua . . . 6
- 4.2 Implementaziao . . . 6
  - 4.2.1 InvertedIteratorAdapter . . . 6
  - 4.2.2 Comparator . . . 7
  - 4.2.3 Main . . . 7
- 4.3 Exekuzioaren emaitza . . . 7

5 GitHub biltegia **8**

1 Simple Factory

1.1 UML diseinua



Irudia 1: Simple Factory UML diseinu berria

1.2 Diseinuan egindako aldaketak

Aplikazioaren diseinua hobetzeko eta aurreko ahultasunak ezabatzeko, honako aldaketa hauek egin ditugu:

- **Klase berria sortu:** `SymptomFactory` izeneko klase berri bat gehitu diogu sistemari.
- **Kodea lekualdatu:** Lehen `Covid19Pacient` eta `Medicament` klaseetan errepikatuta zegoen `createSymptom` metodoa ezabatu dugu, eta `SymptomFactory` klasean sartu dugu metodo publiko gisa.
- **Deiak eguneratu:** Orain, `Covid19Pacient` eta `Medicament` klaseek sintoma bat gehitu behar dutenean, zuzenean `SymptomFactory` klaseari eskatzen diote sintoma hori sortzeko.

Aldaketa hauen bidez, sintomak sortzeko logika leku bakarrean zentralizatzea lortu dugu, kode-bikoiztasuna ezabatuz.

1.3 Inplementazioa: "mareos"sintoma berria gehitzea

Aplikazioaren inplementazioan “mareos” sintoma berria 1 inpaktuarekin gehitzeko, `SymptomFactory` klaseko barne-zerrendak eguneratu ditugu. Zehazki, honako aldaketa hauek egin dira:

- `impact1` zerrendan `"mareos"` string-a gehitu behar da, bere inpaktua 1 izan dadin.
- Bere larritasun-indizea (`severityIndex`) `1.0` bezala ezarri da `index1` zerrendan.
- Sintoma motari dagokionez, `digestiveSymptom` zerrendan sartu dugu, sintoma digestibo gisa sailkatuz.

Hona hemen `SymptomFactory` klasean aldatutako kode zatia:

```
List<String> impact1 = Arrays.asList("nauseas", "vomitos",
    "congestion nasal", "diarrea", "hemoptisis",
    "congestion conjuntival", "mareos");

List<Double> index1 = Arrays.asList(5.0, 4.8, 3.7, 0.9, 0.8,
   0.0, 1.0);

List<String> digestiveSymptom = Arrays.asList("nauseas",
    "vomitos", "diarrea", "mareos");
```

1.4 Symptom objektuen bakartasuna

`Covid19Pacient` eta `Medicament` klaseek erabiltzen dituzten `Symptom` objektuak bakarrak izateko objektu hori `HashMap` batean gorde dugu.

- `HashMap` bat definitu dugu (`symptomCache`) jada sortu diren sintomak gordetzeko. Gakoa sintomaren izena da, eta balioa `Symptom` objektua.
- `createSymptom` metodoaren hasieran, `HashMap`-ean sintoma hori jada badagoen begiratzen da. Egotekotan, instantzia hori itzultzen da berririk sortu gabe.
- Objektua `HashMap`-ean ez badago, sintoma berria sortzen da, eta itzuli aurretik bertan gordetzen da hurrengo deietan berrerabiltzeko.

2 Observer patroia

2.1 Lehen prototipoa

2.1.1 Observer patroiaren inplementazioa

Lehen prototipo honetan, paziente baten sintomak aldatzen direnean behatzaileei (*Observer*) jakinarazteko mekanismoa inplementatzen da.

`Covid19Pacient` klasea (Observable Subjektua)

- `java.util.Observable` klasetik hedatzen da (`extends Observable`).
- Sintoma bat gehitzen denean (`addSymptomByName`) edo ezabatzen denean (`removeSymptomByName`), egoera aldatu dela adierazten da eta behatzaileei jakinarazten zaie: **setChanged()** eta **notifyObservers()** erabilita

`PacientObserverGUI` klasea (Observer Behatzailea)

- `java.util.Observer` interfazea inplementatzen du (`implements Observer`).
- Eraikitzailean jasotako `Observable` objektuari harpidetzen zaio: **obs.addObserver(this**
- `update(Observable o, Object arg)` metodoa eguneratzen da, pazientearen egoera berria jasotzeko eta interfaze grafikoko `symptomLabel` etiketa HTML bidez berriz marrazteko:

`PacientSymptomGUI` **klasea (Kontrolatzailea)**

- Erabiltzaileak interfaze grafikoan botoiak sakatzen dituenean, pazientearen `addSymptomByName` edo `removeSymptomByName` metodoak deitzen dira.

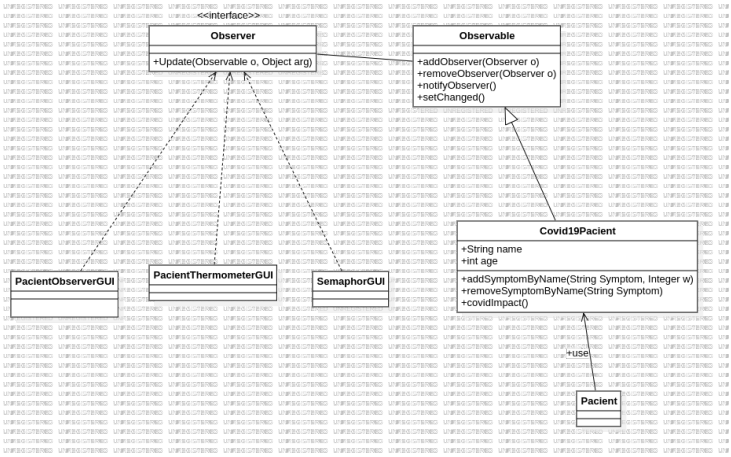
2.2 Bigarren prototipoa

2.2.1 PacientThermometerGUI klasearen inplementazioa

Prototipo honetan behatzaile berri bat gehitzen da: `PacientThermometerGUI`. Interfaze honek pazientearen `covidImpact()` balioa termometro grafiko batean eguneratzen du gertaera bat jasotzen duen bakoitzean.

- `implements Observer` hitz-gakoa gehitzen da.
- Eraikitzailean harpidetza egiten da: obs.addObserver(this)
- `update` metodoaren inplementazioa:

2.2.2 UML diagrama hedatua



Irudia 2: Observer UML diseinu berria.

3 Adapter patroia

3.1 Covid19PacientTableModelAdapter egokitzailea

3.1.1 Inplementazioa

`Covid19PacientTableModelAdapter` klasean beharrezko kodea gehitu dugu. Klase honek **Adapter** patroia inplementatzen du, `AbstractTableModel` hedatuz eta `Covid19Pacient` baten datuak taularen formatura egokituz.

Honako metodo hauek osatu dira:

- `getColumnCount()`: 2 itzultzen du.
- `getColumnName(int i)`: `columnNames` array-tik izena itzultzen du.

- `getRowCount()`: Pazientearen sintoma kopurua itzultzen du `size()` erabiliz.
- `getValueAt(int row, int col)`: `pacient.getSymptoms()` indizerik ez duenez, iteradore bat erabili da (`Iterator<Symptom>`) eskatutako errenkadara iritsi arte. Ondoren, 0 zutabearentzat izena itzultzen da, eta 1 zutabearentzat sintomaren pisua.

3.1.2 Emaitzaren egiaztapena

Aplikazioa `Main` klasetik exekutatzean, interfazeko taulan "disnea", "cefalea" eta "astenia" sintomak ondo zerrendatzen direla konprobatu da, dagozkien pisu edo larritasunekin. Taularen formatua eta datuak zuzen kargatzen dira GUI-an.

3.2 Hainbat pazienteren bistaratzea

Aplikazioak paziente anitzekin ondo funtzionatzen duela egiaztatzeko, `Main` klasea eguneratu dugu paziente berri bat sortuz eta bigarren interfaze grafiko bat instantziatuz.

```
1  public class Main {
2
3    public static void main(String[] args) {
4      Covid19Pacient pacient=new Covid19Pacient("aitor", 35);
5      Covid19Pacient pacient2 = new Covid19Pacient("Mikel", 45);
6
7      pacient.addSymptomByName("disnea", 2);
8      pacient.addSymptomByName("cefalea", 1);
9      pacient.addSymptomByName("astenia", 3);
10     pacient2.addSymptomByName("fiebre", 2);
11     pacient2.addSymptomByName("tos seca", 3);
12     pacient2.addSymptomByName("diarrea", 1);
13
14     ShowPacientTableGUI gui=new ShowPacientTableGUI(pacient);
15     ShowPacientTableGUI gui2 = new ShowPacientTableGUI(pacient2)
           ;
16
17     gui.setPreferredSize(new java.awt.Dimension(300, 200));
18     gui2.setPreferredSize(new java.awt.Dimension(300, 200));
19
20     gui.setVisible(true);
21     gui2.setLocation(320, 0);
22     gui2.setVisible(true);
23   }
24 }
```

3.3 Taulen eguneratze automatikoa (hautazkoa)

Paziente bati sintoma berri bat gehitzean bere `ShowPacientTableGUI` leihoa automatikoki eguneratzeko, honako urrats hauek eman beharko genituzke:

1. **Pazientea `Observable` bihurtu:** `Covid19Pacient` klasea behatua bihurtzeko, `Observable` klasea hedatu behar dugu `extends Observable` gehituz. Bere `addSymptom` metodoan, sintoma berri bat gehitzen den bakoitzean, `setChanged()` eta `notifyObservers()` metodoak deitu behar ditu behatzaile guztiei abisatzeko.

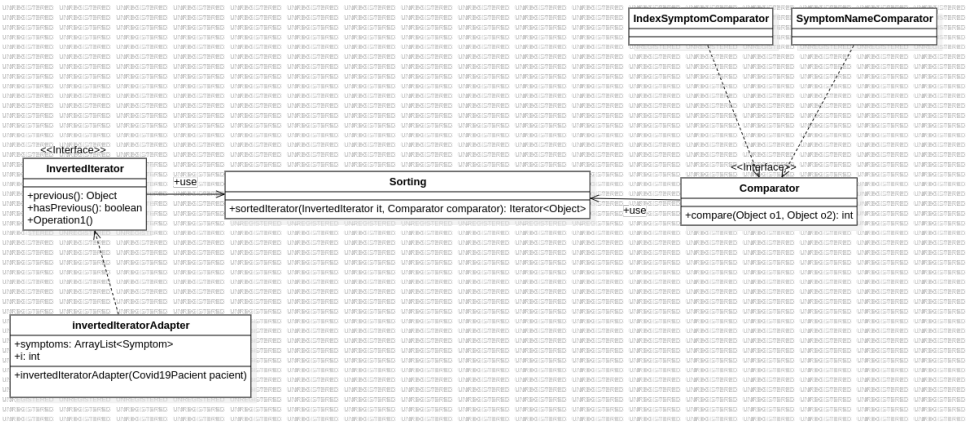
2. **Interfazea `Observer` bihurtu:** `ShowPacientTableGUI` klaseak `Observer` interfazea inplementatu behar du `implements Observer` gehituz.

3. **Behatzailea erregistratu:** GUI-aren eraikitzailean, leihoak bere burua pazientearen behatzaile gisa erregistratu behar du (`pacient.addObserver(this)`).

4. `update` **metodoa inplementatu:** `ShowPacientTableGUI` klasean `update` metodoa gainidatzi behar da. Metodo horren barruan, taularen ereduari datuak aldatu direla jakinaraziko zaio.

4 Adapter eta Iterator patroiak

4.1 UML diseinua



Irudia 3: Observer UML diseinu berria.

4.2 Implementaziao

Atal honetan, paziente baten sintomak ordenatzeko bi diseinu-patroi erabiltzen dira: **Adapter** eta **Strategy**. Horretarako, Sorting izeneko kanpoko klase bat erabiltzen da. Klase horrek elementuak ordenatzeko aukera ematen du, baina ezin da haren kodea aldatu.

4.2.1 InvertedIteratorAdapter

`InvertedIteratorAdapter` klaseak `Set<Symptom>` egitura `Sorting` klaseak behar duen `InvertedIterator` interfazearekin bateragarri egiten du. Horretarako, sintomak `ArrayList` batean gordetzen ditu eta `goLast()`, `hasPrevious()` eta `previous()` metodoak inplementatzen ditu, elementuak azkenetik lehenengora zeharkatzeko, `Sorting` klasearen kodea aldatu gabe.

4.2.2 Comparator

Ordenazio-irizpide desberdinak definitzeko, **Strategy** diseinu-patroia erabiltzen da, `Comparator<Object` interfazearen bidez. Horrela, konparazio-irizpidea ordenazio-algoritmotik bereizten da, eta ordenatzeko modua erraz alda daiteke.

Bi konparatzaile inplementatzen dira:

- **SymptomNameComparator:** sintomak alfabetikoki ordenatzen ditu izenaren arabera, `getName()` eta `compareTo()` metodoak erabiliz.
- **IndexSymptomComparator:** sintomak larritasunaren arabera ordenatzen ditu, `severityIndex` atributua erabiltzen eta `index1 eta index2` arteko kenketa egiten.

Horrela, ordenazio-irizpidea alda daiteke **Sorting** klasearen inplementazioa aldatu beharrik gabe.

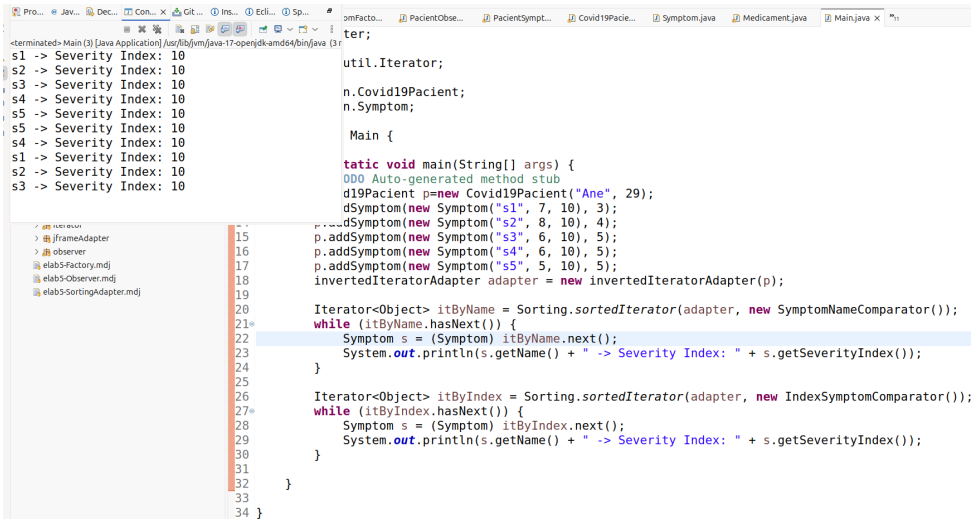
4.2.3 Main

Azkenik, `Main` klasean inplementazio guztien funtzionamendua egiaztatzen da. Horretarako, `Covid19Pacient` objektu bat sortzen da sintoma batzuekin, eta `InvertedIteratorAdapter` erabiltzen da sintomak alderantzizko ordenan zeharkatzeko.

Ondoren, `Sorting.sortedIterator(adapter, comparator)` metodo estatikoa deitzen da, egokitzailea eta ordenazio-irizpidea zehazten dituen konparatzailea pasatuz. Bi estrategiak probatzeko, `SymptomNameComparator` eta `IndexSymptomComparator` erabiltzen dira. Azkenik, emaitzak pantailan inprimatzen dira.

Horrela, bi diseinu-patroiak batera erabiltzen dira: **Adapter** patroiak egituren arteko bateragarritasuna bermatzen du, eta **Strategy** patroiak ordenazio-irizpidea aldatzea ahalbidetzen du, algoritmo nagusia aldatu gabe.

4.3 Exekuzioaren emaitza



Irudia 4: Observer UML diseinu berria.

5 GitHub biltegia

https://github.com/erabiltzailea/biltegia